

T07 인증 구현 설명서

- 결과물: <https://ex-6-seven.vercel.app/> (첫 화면은 로그인 화면, 로그인 없이도 이 화면까지는 열림)
- 소스: <https://github.com/LeeHakSan/ex-6> (T06 최종 지점은 `t06-final` 태그로 고정)
- 원본 요청/응답 전체 기록: `docs/t07_evidence_raw.json`

① 무엇으로 붙였나

직접 구현. 비밀번호 해시는 `bcryptjs@2.4.3`, 세션 토큰(JWT) 서명·검증은 `jsonwebtoken@9.0.2`, 쿠키 직렬화는 `cookie@0.6.0` 을 각각 라이브러리로 사용했다(모두 `package.json` 에 버전 고정). 가입/로그인/로그아웃 로직, 세션 만료·무효화 로직, 계정 간 소유권 검사 로직은 직접 작성했다.

② 왜 그걸 골랐나

T06이 이미 Supabase를 쓰고 있어서 **Supabase Auth(GoTrue)** 를 붙이는 것도 함께 검토했다. Supabase Auth를 쓰면 붙이는 작업 자체는 더 짧았을 것이다. 하지만 이 과제의 목적이 "인증을 어떻게 붙였는지 설명하기"이고, Supabase Auth를 쓰면 가입→해시→세션 발급→만료→소유권 검사로 이어지는 과정이 라이브러리 내부에 완전히 숨어서 "어떻게 붙였는지"를 직접 보여줄 수 없다고 판단했다. 그래서 각 단계를 직접 구현하는 쪽을 선택했다.

③ 어디를 어떻게 고쳤나

T06(정적 프론트 + Supabase anon key 직접 호출) 구조에 Vercel 서버리스 함수(`/api`)를 새로 추가하고, 프론트가 Supabase에 직접 붙던 것을 전부 `/api` 호출로 바꿨다.

흐름	소스 위치
가입	<code>api/auth/[action].js</code> (<code>action=signup</code>) → <code>api/_lib/auth.js</code> 의 <code>hashPassword</code>
로그인	<code>api/auth/[action].js</code> (<code>action=login</code>) → <code>verifyPassword</code> , <code>createSession</code> , <code>signToken</code> , <code>setSessionCookie</code>
로그아웃	<code>api/auth/[action].js</code> (<code>action=logout</code>) → <code>revokeSession</code> , <code>clearSessionCookie</code>
자료 조회(계획/할 일/실행기록/내보내기/계정삭제)	<code>api/data/plans.js</code> , <code>api/data/todos.js</code> , <code>api/data/plan-revisions.js</code> , <code>api/data/execution-logs.js</code> , <code>api/data/export.js</code> , <code>api/data/account.js</code> — 전부 <code>api/_lib/auth.js</code> 의 <code>requireAuth(req)</code> 로 세션 확인 후 <code>user_id</code> = 로그인한 사용자 로 필터/검사

DB 변경: `sql/schema-v3-auth.sql` 에서 `users` / `sessions` 테이블 신설, `plans` / `todos` / `execution_logs` / `plan_revisions` 에 `user_id` 컬럼 추가, T06 시절의 익명 전체 허용 RLS 정책을 전부 삭제(잠금). 이후 `/api` 만 `SUPABASE_SERVICE_ROLE_KEY` 로 RLS를 우회해 접근하고, 소유권 판단은 RLS가 아니라 각 API 핸들러 코드에서 직접 한다.

막혔던 지점 하나: 처음엔 `api/data/[...route].js` 하나로 `/api/data/plans/:id` 처럼 경로에 id를 실어 REST 스타일로 짰는데, 배포 후 실제로 두 단계 이상인 경로(`/api/data/plans/:id` , `/api/data/plans/:id/revisions`)가 우리 함수에 전혀 도달하지 못하고 Vercel 자체 404(`x-Vercel-Error: NOT_FOUND`)로 가로채지는 것을 발견했다(계획 수정 기능이 실제로 동작하지 않고 있었다). 원인은 이 배포 환경에서 동적 catch-all 라우트가 2단계 이상 경로를 안정적으로 매칭하지 못하는 것으로 확인되어, id를 경로가 아니라 쿼리스트링(`?id=`)으로 넘기는 방식으로 API를 재구성해 해결했다.

④ 안 열리는 것을 확인한 기록 (다섯 가지)

전체 원본은 `docs/t07_evidence_raw.json` 참고. 대표 요청/응답만 발췌.

1) 비밀번호가 원문으로 저장되지 않음 (카드2)

- 방법: `bcrypt( bcryptjs )`, cost factor 12
- 로그인 요청/응답 어디에도 비밀번호 원문 없음:

```
POST /api/auth/login { "email": "...", "password": "(가려짐, 요청 바디는 HTTPS로 암호화되어 전송됨)"
200 { "email": "alice-...@example.com" } ← 응답에도 비밀번호 없음
```

- 저장된 값 확인: `users.password_hash` 컬럼이 전부 `$2a$12$...` 형태의 bcrypt 해시로 저장되어 있고, 입력한 비밀번호 글자가 그대로 보이지 않는다.
- 같은 비밀번호로 다른 값: 아래 스크린샷에서 `Passw0rd!123` 을 똑같이 쓴 alice/bob 계열 테스트 계정 여러 개(`alice-...` , `bob-...` , `bob2-...` 등)의 `password_hash` 값이 서로 전부 다르다 — bcrypt가 계정마다 다른 salt를 자동으로 붙이기 때문.

☐	id uuid	email text	password_hash text	created_at timestampz	+
☐	67aeef66-2ae9-46aa-922b-2902d2c9	diag-full-flow-1788311671@example.com	\$2a\$12\$03fphkOOG50SJa9y.Oi4ueYz	2026-09-02 01:13:36.038423	
☐	6a383369-fd45-4560-83c4-65160814	empty-dd-check3-1788313851657@ex	\$2a\$12\$8TMfLe2R20qtvykh/DzW.OFA	2026-09-02 01:49:55.672537	
☐	6eab4ef0-97c5-4cbb-9ca2-917d0cbb	bob-1788315148330@example.com	\$2a\$12\$.s1PUkSt/sGEjvtdffo1aupAh1aJ	2026-09-02 02:11:41.055425	
☐	70fd64dc-79f3-41f5-a7a2-f839cac667	bob-1788315183957@example.com	\$2a\$12\$VG2dwSWWq7Aol9ias7IWw.5z	2026-09-02 02:12:14.701451	
☐	8e0d4bd1-0f63-4beb-8a82-b62887d0	test2@test.com	\$2a\$12\$ypuVBVYhBVFiP6G8J986auol	2026-09-02 01:45:25.316346	
☐	8fa8d2a6-5dd1-4809-a52c-c6d4d5af4	diag-plans-test-1788311483@example.	\$2a\$12\$RzXbkT4CFLz1IODQVVzSp.8b	2026-09-02 01:10:28.284198	
☐	a11cb1ac-f988-4d32-a541-7e16ee3d1e	empty-dd-check4-1788313936596@ex	\$2a\$12\$275X/ID9IGx74ITX3DtX.O/5.K	2026-09-02 01:51:20.170658	
☐	a7faa2b8-1863-4da1-aa30-e993bd5eb	empty-dd-check2-1788313814798@ex	\$2a\$12\$0JnWtcDdBPfXPur.KdJ9.71Y	2026-09-02 01:49:18.948628	
☐	a8c008ed-dc9c-4b5f-9468-87ee18a6	alice2-1788314877478@example.com	\$2a\$12\$O7PbHq/DUbQNeo/eEXJOCO5	2026-09-02 02:07:01.450421	
☐	afb9d805-23b7-496b-8483-1f984f4a5	test1@test.com	\$2a\$12\$Vo62Cac7FPBF84nmHLiru7t.	2026-09-02 01:15:50.308595	
☐	b416f497-908c-4827-ba06-5b82f4726	cookie-jar-check-1788313908115@exa	\$2a\$12\$e6GF6×7Pe5U4R7FP2Cy2ru8	2026-09-02 01:50:51.657818	
☐	c953535a-88be-47e5-9b62-f9994a55	alice-1788315183957@example.com	\$2a\$12\$XaN2CGOCKNTEGagLOPCrse	2026-09-02 02:12:07.065569	
☐	cdc458af-0107-4b42-ae98-91ff216893	patch-check-1788314940@example.cc	\$2a\$12\$PLr/MDyWAlpDF3ygyBewOth	2026-09-02 02:08:04.98375	
☐	d997fb0a-40fe-449a-9f29-420b921a4	dropdown-check-1788313461663@exa	\$2a\$12\$CWxiqFgyIW23Uf.cJAYQD.aF5	2026-09-02 01:43:26.46645	
☐	de9034b0-7625-49f5-ad2e-23ad940e	bob2-1788314877478@example.com	\$2a\$12\$igWfUozrR/ahkRYzerY5auEQk	2026-09-02 02:07:03.34349	
☐	e11e802a-b74b-4e62-8691-47f10afb86	test@test.com	\$2a\$12\$2ukSa8pxzvfGz4F3WYjzyOC1	2026-09-02 01:07:47.720556	
☐	e1b49114-6b92-46c6-9c92-baa01d9et	empty-dd-check5-1788313975662@ex	\$2a\$12\$5ID6CqPF6pvVqqWG3NrvoOo	2026-09-02 01:51:59.728695	
☐	fc536fcc-dfee-43f6-9a84-c8fc1cb9ec	empty-dd-check-1788313790653@exa	\$2a\$12\$FdxDIU6UXUQ7CZmzoxnfjuVH	2026-09-02 01:48:55.051536	
☐	ff042f27-fd46-495f-b719-d5ea92a61a	bob2-1788315226016@example.com	\$2a\$12\$6.oW6GdGloGz4xowXDSZHOI	2026-09-02 02:12:51.741233	

2) 세션 만료·로그아웃 후 재요청 거절 (카드3)

사람을 알아보는 값: 토큰(JWT). 쿠키 이름 `session`, `httpOnly; Secure; SameSite=Lax`, 만료 7일. 로그아웃 시 서버 `sessions` 테이블의 `revoked_at` 을 찍어 즉시 무효화(순수 stateless JWT라면 여기서 막히지 않음 — 그래서 세션 레코드를 별도로 둠).

```
로그아웃 전 GET /api/data/plans (세션 쿠키 0) → 200
POST /api/auth/logout → 200 { "ok": true }
로그아웃 후 GET /api/data/plans (같은 쿠키 값 재사용) → 401 { "error": "not authenticated" }
```

같은 주소(`/api/data/plans`)·같은 방식(GET)·같은 쿠키 값인데 로그아웃 여부만 다르다.

3) 로그인 없이 자료에 접근 불가 (카드1)

```
GET /api/data/plans (쿠키 없음) → 401 { "error": "not authenticated" }
```

브라우저에서도 로그인하지 않은 채 결과물 주소를 열면 로그인 화면만 보이고 자료 화면은 렌더링되지 않는다 (`js/main.js` 의 `checkSession()` 이 `/api/auth/me` 실패 시 `showAuthScreen()` 만 호출).

로그인 실패 메시지도 계정 존재 여부와 무관하게 동일:

```
존재하는 계정 + 틀린 비번 → 401 { "error": "이메일 또는 비밀번호가 올바르지 않습니다." }
존재하지 않는 계정      → 401 { "error": "이메일 또는 비밀번호가 올바르지 않습니다." }
```

4) 계정 간 읽기·수정·삭제 양방향 차단 (카드4)

Alice-Bob 두 계정을 만들어 각자 계획/할 일을 하나씩 넣고 서로의 자료를 공격:

```
Bob → Alice 계획 수정 PATCH /api/data/plans?id=<Alice의 id> (Bob 세션) → 404 { "error": "no"
Bob → Alice 수정이력 조회 GET /api/data/plan-revisions?plan_id=<Alice의 id> (Bob 세션) → 404
Bob → Alice 할일목록 조회 GET /api/data/todos?plan_id=<Alice의 id> (Bob 세션) → 404
Bob → Alice 할일 삭제 PATCH /api/data/todos?id=<Alice 할일 id>&action=delete (Bob 세션) → 404
```

반대 방향(Alice → Bob)도 전부 동일하게 404. 공격 시도 전후로 상대방 자료 건수·내용은 그대로였다 (`stillOriginalTitle: true`, `bobTodoStillExists/aliceTodoStillExists: true`). 거절을 만드는 소스: `api/data/plans.js` · `api/data/todos.js` · `api/data/plan-revisions.js` 의 매 쿼리에 붙는 `.eq('user_id', userId)` (요청자의 세션에서 얻은 id로만 필터링, 요청 바디·쿼리에 실린 다른 사용자 id는 절대 신뢰하지 않음).

5) 목록에 남의 자료 미포함 + 아이디 스푸핑 방어 (카드4)

```
Alice 목록에 Bob 계획 포함? → false
Bob 목록에 Alice 계획 포함? → false
```

요청 본문에 다른 사용자 id를 끼워 넣어도 무시되고 실제 세션 주인 것으로 생성됨:

POST /api/data/todos { ..., "user_id": "bob-id-를-사칭" } (Alice 세션)
→ 201, 실제 생성된 todo.user_id == Alice의 진짜 id (끼워 넣은 값 무시됨)

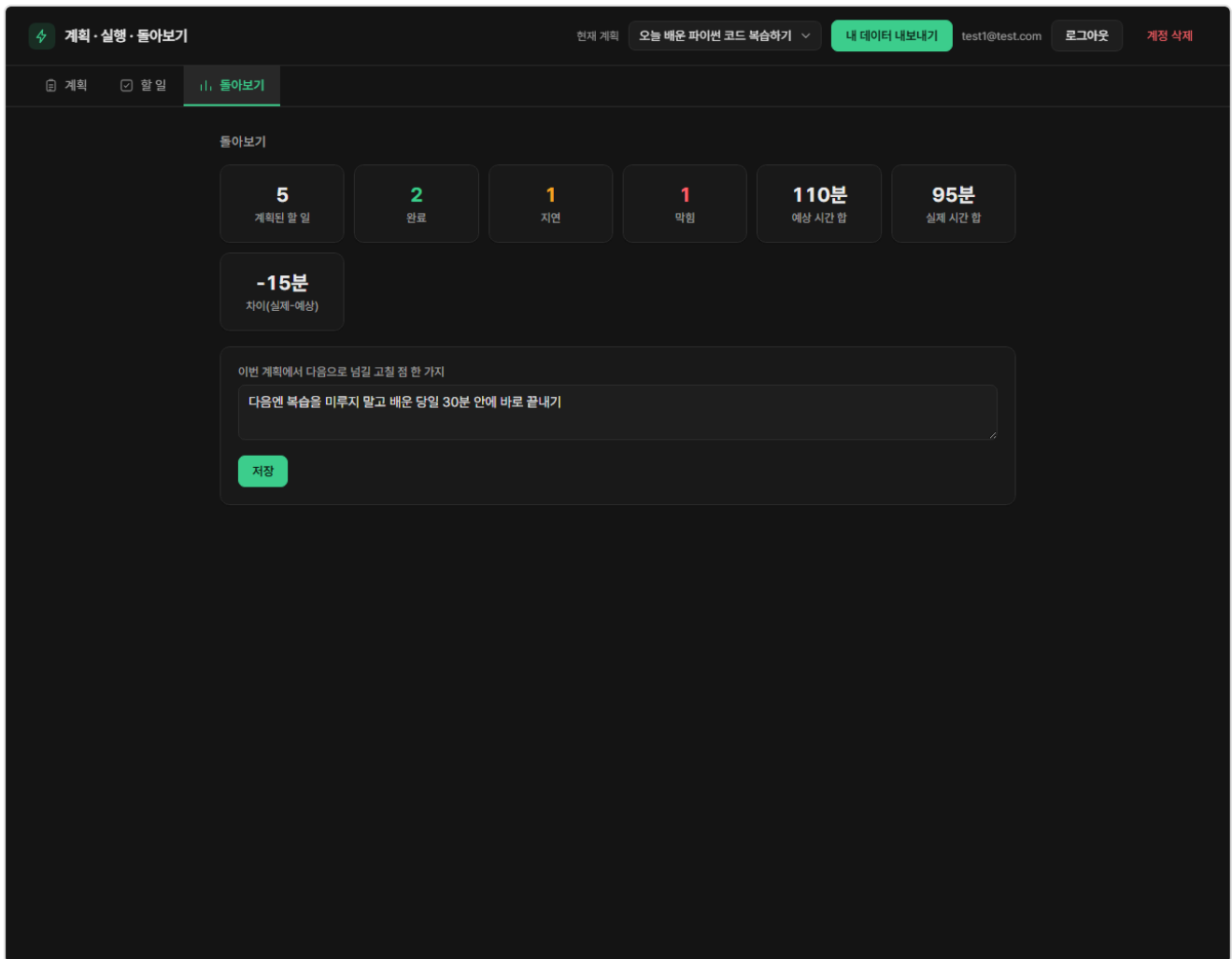
부록 — 카드 5, 1일차 실사용 기록

5일 실사용은 실제로 다른 날짜에 걸쳐야 하는 항목이라 지어낼 수 없다. 그래서 지어내는 대신 **1일차(2026-09-02)**만 실제 계정(test1@test.com)으로 진짜 사용하고, 나머지 4일은 못 채웠음을 그대로 밝힌다.

- 1일차 질문: "오늘 계획한 할 일을 얼마나 완료했는가?"
- 관찰 지표: 완료한 할 일 개수 (단위: 건)
- 계산 규칙: 그날 상태가 "완료"로 바뀐 할 일 수를 그대로 센다

오늘 실제로 할 일 하나를 완료 처리하고 실행 기록을 추가한 결과(완료 2건, 실제 시간 95분으로 반영됨):

The screenshot shows a web application interface for task management. At the top, there's a navigation bar with tabs: '계획' (Plan), '할 일' (Tasks), and '돌아보기' (Review). The '할 일' tab is active. Below the navigation bar, there's a header area with '새 할 일 만들기' (Create new task) and a search bar. The main content area displays a list of tasks. One task, '클래스와 객체지향 기초 복습' (Review of classes and object-oriented basics), is highlighted. A modal window is open over this task, titled '실행 기록 — 클래스와 객체지향 기초 복습' (Execution record — Review of classes and object-oriented basics). The modal contains a table of execution records with columns for '시작 시각' (Start time), '종료 시각' (End time), and '완료 여부' (Completed status). The first record shows a start time of '2026. 9. 2. 12시 12분 0초' and an end time of '2026. 9. 2. 12시 32분 0초'. The second record shows a start time of '2026. 8. 31. 20시 0분 0초' and an end time of '2026. 8. 31. 20시 15분 0초'. The modal also includes a '기록 추가' (Add record) button and a '기록 저장' (Save record) button.



⑤ AI와 나

- **AI에게 맡긴 일:** 인증 아키텍처 설계 초안, 백엔드(/api) 전체 코드 작성, DB 마이그레이션 SQL 작성, 배포 후 자동화된 증거 수집(가입/로그인/로그아웃/세션 만료/계정 간 차단 테스트) 및 이 설명서 초안 작성.
- **내가 직접 판단한 일:** 인증 방식(직접 구현 vs Supabase Auth)을 직접 구현으로 최종 결정, 실제 계정 (test1@test.com) 지정 및 T06 데이터 이관 대상 결정, UI/UX 문제(로그인·회원가입 폼 동시 노출, 드롭다운 스타일, 회원가입 후 처리 방식) 직접 발견하고 수정 방향 지시.
- **AI 제안을 따르지 않은 일:** AI가 처음 추천한 Supabase Auth 대신 직접 구현(bcrypt+JWT)을 선택. 또한 AI가 회원가입 성공 시 자동 로그인시키도록 짰던 것을, 성공 문구를 먼저 보여주고 로그인 화면으로 돌아가게 바꾸도록 지시함.

⑥ 아직 못 막은 것

- **무차별 대입(brute force) 공격 방어 없음:** 로그인 실패 횟수를 세거나 잠그는 로직이 없다. 같은 이메일로 비밀번호를 계속 시도해도 막히지 않는다 — 공격자가 흔한 비밀번호 목록으로 계정을 뚫을 위험이 있다.
- **비밀번호 재설정 기능 없음:** 비밀번호를 잊으면 복구할 방법이 없다(계정 삭제 후 재가입만 가능). 실제 서비스라면 이메일 인증 기반 재설정이 필요하다.

- **가입 시 이메일 소유 확인(이메일 인증) 없음:** 아무 이메일 주소로나 가입이 가능해 존재하지 않는/타인의 이메일로 계정을 만들 수 있다.
- **로그인 시도 로그(감사 로그) 없음:** 누가 언제 로그인/실패했는지 서버에 별도로 기록하지 않아, 이상 접근을 사후에 추적하기 어렵다.